

Constructors in C# with Examples

In this article, I am going to discuss **Constructors in C#** with Examples. Please read our previous article before proceeding to this article where we discussed [how to create classes and objects in C#](#) with examples. Object-Oriented Programming is all about writing code inside a class. A class is a collection of members like fields, methods, constructors, etc. Inside a class one of the most important members present is called Constructor.

What is a Constructor in C#?

It is a special method present inside a class responsible for initializing the variables of that class. We will come to this point later part of this article.

The name of the constructor method is exactly the same name as the class in which it was present. You cannot change the name. If your class name is Employee, then the name of the constructor method is going to be Employee, and if your class name is Student, then the constructor name is also going to be Student.

The constructor method does not return any value. That means it is a non-value returning method. Generally, methods are of two types i.e. value returning and non-value returning and constructors are purely non-value returning. That is, they never return any value.

Example to Understand Constructor in C#

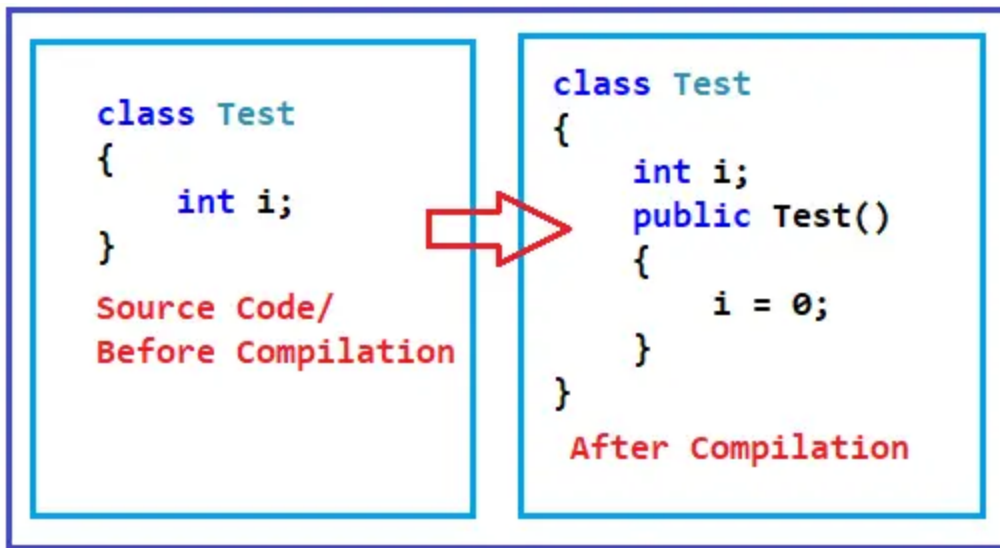
Each and every class requires this constructor if we want to create the instance of the class. If we don't have a constructor, then we cannot create an instance of the class. At this point, you have one doubt, earlier we defined many classes but we never used a constructor, but still, we are able to create the instance of the class, how? Let us clarify this doubt. Suppose, we have a class as follows:

```
class Test
{
    int i;
}
```

Then, we create an instance of the above Test class somewhere in our application as follows:

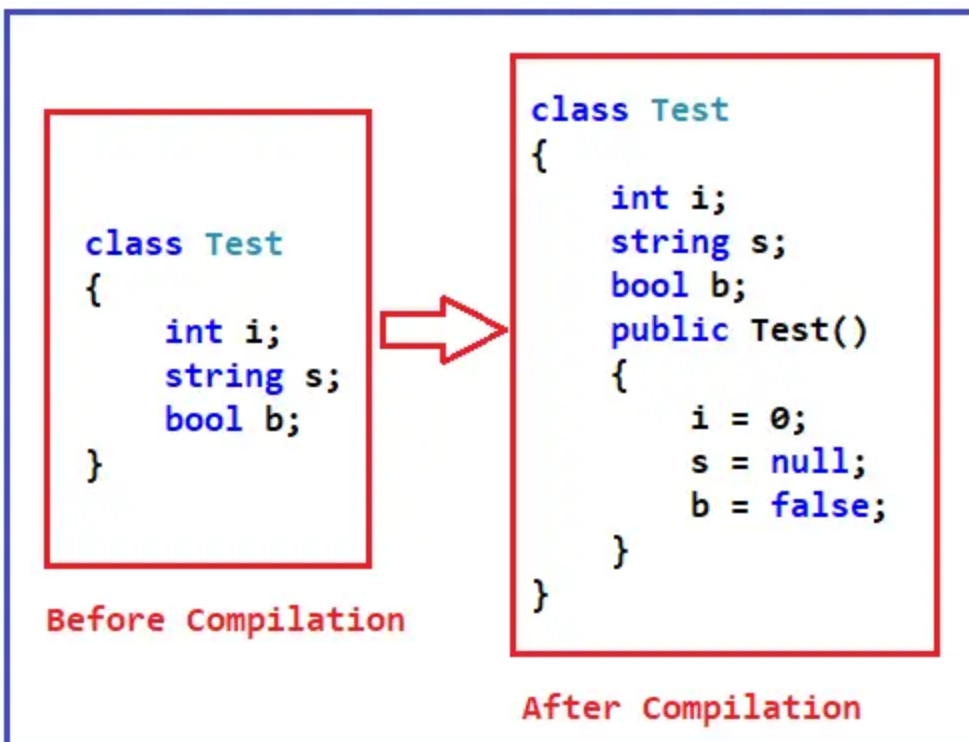
```
Test obj = new Test();
```

Is the above statement valid? Yes, it is valid. The reason is that it is the responsibility of a programmer to define a constructor under his class and if he/she fails to do, on behalf of the programmer an implicit constructor gets defined in that class by the compiler. For a better understanding, please have a look at the below diagram which shows the code before and after compilation.



You can see here that after compilation, the compiler adds the public constructor to the class and initializes the variable and this is the responsibility of a constructor i.e. initializing the variables of that class. Here, it is initializing the variable with 0. If a class variable is initialized implicitly means that is done by a constructor.

Every variable we declared inside a class and every field we declared inside a class has a default value. All numeric types are initialized with 0, Boolean types initialized with false, and string and object types initialized with null. For a better understanding, please have a look at the below image.



Like this, the initialization is performed for each and all variables present in the class and this is the responsibility of the constructor. That is why a constructor is very important for us inside a class.

We are not assigning a value, but a value is coming there means someone has assigned the value to these variables. So, who is going to do that? The constructor is going to do that. And this constructor is not defined by us. Then who defined this constructor? The compiler defined this constructor for us. And we call this an Implicit Constructor. And if we defined the same thing, then it is called an explicit constructor.

Points to Remember while working with Constructors in C#:

1. Implicitly Defined Constructors are parameter less and these constructors are also known as Default Constructors. This is because they are used to initialize the variables with default values.
2. Implicitly Defined Constructors are public. If you see in our example, we define the class Test with a default access specifier but the constructor is public which is generated by the compiler.
3. We can also define a constructor under the class and if we define it, we can call it an Explicit Constructor and an Explicit Constructor can be parameter less and parameterized also.

Example to Understand Implicitly Constructor in C#:

In the below example, we are creating a class with three variables and in the class, we have not defined any constructor explicitly. So, here compiler will provide the implicit constructor and will initialize the variables with the default value. Then from the Main method, we create an instance of the class and print the values of the variables and it should print the default values based on the variable type. For example, for int, the default value is 0, for bool the default value is false, and for string or object the default is null. And this default initialization is done by the implicit constructor which is given by the compiler.

```
using System;
namespace ConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Test obj = new Test();
            Console.WriteLine($"i = {obj.i}");
            Console.WriteLine($"b = {obj.b}");
            //value null will be printed, so here we checking the null
            if (obj.s == null)
            {
                Console.WriteLine("s = null");
            }
            Console.ReadKey();
        }
    }
    class Test
    {
        public int i;
        public bool b;
        public string s;
    }
}
```

Output:

```
i = 0
b = False
s = null
```

How to Define the Constructor Explicitly in C#?

We can also define the constructor explicitly in C#. The following is the explicit constructor syntax.

```
[<modifiers>] <class name> ([<Parameter List>])  
{  
    //Statements  
}
```

Whenever we are creating an instance, there will be a call to the class constructor. For a better understanding, please have a look at the below example. Here, we defined one parameter less constructor explicitly, and then from the Main method, we create an instance. When we create the instance, it will make a call to the constructor, and the statements written inside the constructor will be executed. In this case, it will execute the print statement in the console.

```
using System;  
namespace ConstructorDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            ExplicitConstructor obj = new ExplicitConstructor();  
  
            Console.ReadKey();  
        }  
    }  
    class ExplicitConstructor  
    {  
        public ExplicitConstructor()  
        {  
            Console.WriteLine("Explicit Constructor is Called!");  
        }  
    }  
}
```

Output: Explicit Constructor is Called!

One more important point that you need to remember is, how many instances you created, and that many times the constructor is called for us. Let us prove this. Please modify the example code as follows. Here, I am creating the instance four times and it should and must call the constructor 4 times and we should see the print statement four times in the console window.

```
using System;  
namespace ConstructorDemo  
{  
    class Program
```

```

{
    static void Main(string[] args)
    {
        ExplicitConstructor obj1 = new ExplicitConstructor();
        ExplicitConstructor obj2 = new ExplicitConstructor();
        ExplicitConstructor obj3 = new ExplicitConstructor();
        ExplicitConstructor obj4 = new ExplicitConstructor();
        Console.ReadKey();
    }
}
class ExplicitConstructor
{
    public ExplicitConstructor()
    {
        Console.WriteLine("Explicit Constructor is Called!");
    }
}
}

```

Output:

```

Explicit Constructor is Called!
Explicit Constructor is Called!
Explicit Constructor is Called!
Explicit Constructor is Called!

```

We should not use the word Implicitly while calling the constructor in C#, why?

See, if we are not defining any constructor explicitly, then the compiler will provide the constructor which is called Implicitly Constructor. See, the following example. If you move the mouse pointer over the Test class, then you will see the following. Here, Test is a class present under the ConstructorDemo namespace.

```

using System;
namespace ConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Test obj = new Test();
        }
    }
    class Test
    {
        public int i;
        public bool b;
        public string s;
    }
}

```

Here, Test is a class belongs to ConstructorDemo namespace

Now, move the mouse pointer to Test() as shown in the below image. Here, the first Test is the class name and the second Test() is the constructor. That means we are calling the constructor explicitly.

```

namespace ConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Test obj = new Test();
            Console.ReadKey();
        }
    }
    class Test
    {
        public int i;
        public bool b;
        public string s;
    }
}

```

Making call to Constructor

Test is the class name

Here, we are explicitly making a call to the constructor and when we call the constructor, the implicit constructor which is provided by the compiler is called and will initialize the variables.

Now coming to the ExplicitConstructor example, we are also doing the same thing. Please have a look at the below example. If you move the mouse pointer over the ExplicitConstructor class, then you will see the following. Here, ExplicitConstructor is a class present under the ConstructorDemo namespace.

```
namespace ConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ExplicitConstructor obj1 = new ExplicitConstructor();
            ExplicitCon class ConstructorDemo.ExplicitConstructor uctor();
            ExplicitCon uctor();
            ExplicitConstructor obj4 = new ExplicitConstructor();

            Console.ReadKey();
        }
    }
    class ExplicitConstructor
    {
        public ExplicitConstructor()
        {
            Console.WriteLine("Explicit Constructor is Called!");
        }
    }
}
```

Here, ExplicitConstructor is a class present under ConstructorDemo namespace

Now, move the mouse pointer to ExplicitConstructor() as shown in the below image. Here, the first ExplicitConstructor is the class name and the second ExplicitConstructor() is the constructor. That means we are calling the constructor explicitly.

```

using System;
namespace ConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ExplicitConstructor obj1 = new ExplicitConstructor();
            ExplicitConstructor obj2 = new ExplicitConstructor();
            ExplicitConstructor obj3 = new ExplicitConstructor();
            ExplicitConstructor obj4 = new ExplicitConstructor();

            Console.ReadKey();
        }
    }
}

class ExplicitConstructor
{
    public ExplicitConstructor()
    {
        Console.WriteLine("Explicit Constructor is Called!");
    }
}

```

Class Name

Making Call to Constructor

Here, we are explicitly making a call to the constructor and when we call the constructor, the explicit constructor which is provided by us is called and will initialize the variables. So, here you might be confused with terms. Defining and calling.

Defining and Calling Constructor in C#:

Defining: Defining a constructor means implementing a constructor in your class. Defining can be two types i.e. Implicit and Explicit. Implicit means the compiler will define the constructor. Explicit means we as a programmer define the constructor. The following code shows defining a constructor explicitly.

```

class ExplicitConstructor
{
    //Defining Constructor Explicitly
    public ExplicitConstructor()
    {
        Console.WriteLine("Explicit Constructor is Called!");
    }
}

```

Calling: Whenever we are creating the instance, we are calling the constructor. Calling is Explicit. We should only call. There is no implicit call to the constructor. For a better understanding, please have a look at the below code.

```

class Program
{
    static void Main(string[] args)
    {
        //Calling Constructor Explicitly
        ExplicitConstructor obj1 = new ExplicitConstructor();
        ExplicitConstructor obj2 = new ExplicitConstructor();
        ExplicitConstructor obj3 = new ExplicitConstructor();
        ExplicitConstructor obj4 = new ExplicitConstructor();

        Console.ReadKey();
    }
}

```

The calling should be done explicitly by us. That may be an implicit or explicit constructor but calling the constructor should be explicit.

Frequently Asked Interview Questions:

What is a Constructor in C#?

In simple words, we can define the constructors in C# are the special types of methods of a class that are executed whenever we create an instance (object) of that class. The Constructors are responsible for two things. One is the object initialization and the other one is memory allocation. The role of the new keyword is to create the object and the role of the constructor is to initialize the variables.

What are the rules to follow while working with C# Constructor?

1. The constructor's name should be the same as the class name.
2. It should not contain a return type even void also.
3. As part of the constructor body return statement with a value is not allowed.

What does a Constructor have in C#?

1. It can have all five accessibility modifiers i.e. public, private, protected, etc.
2. The constructor can be parameterless or parameterized.
3. It can have a throws clause which means we can throw an exception from the constructor.
4. The constructor can have logic, as part of logic it can have all C#.NET legal statements except return statements with value.
5. We can place a return; in the constructor.

Syntax:

```
class Sample
{
    //Defining Constructor
    public Sample()
    {
        //All .NET Legal Statemenets are allowed
        //Except return statement with value
    }
}
```

Can we define a method with the same class name in C#?

No, it is not allowed to define a method with the same class name in C#. It will give you a compile-time error. For a better understanding, please have a look at the below image.

```
class ExplicitConstructor
{
    public ExplicitConstructor()
    {
        Console.WriteLine("Explicit Constructor is Called!");
    }

    public void ExplicitConstructor(int x, int Y)
    {
    }
}
```

void ConstructorDemo.ExplicitConstructor.ExplicitConstructor(int x, int Y)

'ExplicitConstructor': member names cannot be the same as their enclosing type

In the next article, I am going to discuss the various [Types of Constructors in C#](#) with Examples. Here, in this article, I try to explain the basic concepts of Constructors in C# with Examples. I hope you enjoy this article. Please give your feedback, suggestions, and questions about this article in the comment section.